mathematics

MDPI

Article

Decentralized Federated Learning with Prototype Exchange

Lu Qi, Haoze Chen *, Hongliang Zou, Shaohua Chen, Xiaoying Zhang and Hongyan Chen

College of Modern Science and Technology, China Jiliang University, Yiwu 322002, China; given_qi@163.com or given.l.qi@cjlu.edu.cn (L.Q.); s20030810013@cjlu.edu.cn (H.Z.); l191800030@cjlu.edu.cn (S.C.); zxy_109@126.com or zhangxiaoying@cjlu.edu.cn (X.Z.); bbchy@163.com or 04a0105065@cjlu.edu.cn (H.C.)

* Correspondence: haozechen@cjlu.edu.cn

Abstract: As AI applications become increasingly integrated into daily life, protecting user privacy while enabling collaborative model training has become a crucial challenge, especially in decentralized edge computing environments. Traditional federated learning (FL) approaches, which rely on centralized model aggregation, struggle in such settings due to bandwidth limitations, data heterogeneity, and varying device capabilities among edge nodes. To address these issues, we propose PearFL, a decentralized FL framework that enhances collaboration and model generalization by introducing prototype exchange mechanisms. PearFL allows each client to share lightweight prototype information with its neighbors, minimizing communication overhead and improving model consistency across distributed devices. Experimental evaluations on benchmark datasets, including MNIST, CIFAR-10, and CIFAR-100, demonstrate that PearFL achieves superior communication efficiency, convergence speed, and accuracy compared to conventional FL methods. These results confirm PearFL's efficacy as a scalable solution for decentralized learning in heterogeneous and resource-constrained environments.

Keywords: federated learning; distributed machine learning; prototype exchange

MSC: 68W15

check for updates

Academic Editor: Raymond Lee

Received: 15 November 2024
Revised: 4 January 2025
Accepted: 9 January 2025
Published: 12 January 2025

Citation: Qi, L.; Chen, H.; Zou, H.; Chen, S.; Zhang, X.; Chen, H. Decentralized Federated Learning with Prototype Exchange. *Mathematics* **2025**, *13*, 237. https://doi.org/10.3390/math13020237

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In recent years, a growing number of AI applications have become deeply embedded in our daily lives, including facial recognition and intelligent transportation systems. These applications leverage the power of deep learning and extensive data collection, drawing from sources like IoT devices, mobile phones, and sensors. Despite these advancements, a significant portion of data remains untapped, holding potential to further enhance existing AI applications. However, utilizing these distributed data presents challenges, as traditional centralized data collection and model training pipelines may no longer be feasible. This shift is largely driven by increasingly stringent data protection regulations, which mandate that sensitive data must not leave trusted user environments.

The advent of edge computing allows learning tasks to be distributed to edge devices (such as users' smartphones), reducing reliance on cloud computing resources. Within this context, federated learning (FL) has been introduced as a means to prevent user data from being transferred to remote servers, thereby mitigating exposure risks and efficiently harnessing computational power at the network edge.

In conventional distributed model training paradigms, such as FL and large language models (LLMs), a centralized parameter server aggregates model parameters and synchronizes model updates. Under this framework, each participant in training sends their model

parameters to a central server. Training participants then wait for the server to aggregate a sufficient number of parameters before receiving the updated global model.

However, applying this paradigm in edge computing environments introduces several challenges: (1) From a network perspective, there is a substantial mismatch between the size of model parameters and the bandwidth constraints of edge networks. For instance, VGG [1] and ResNet50 [2] models are approximately 500 MB and 100 MB, respectively, while mobile edge network bandwidths are typically only 10–50 Mbps [3,4], leading to significant delays in parameter transmission. (2) From the user's perspective, prolonged data transmission results in considerable battery drain and data usage. Wireless networks are inherently best-effort, and poor network conditions can lead to a higher probability of packet loss and more frequent retransmissions, increasing communication energy consumption and reducing device battery life. (3) From the device perspective, computing power varies among devices, meaning that clients with lower computational capacity may become bottlenecks, hindering overall system training progress. Inspired by peer-to-peer architectures, we turn our focus to decentralized federated learning, where each client only shares model parameters with nearby peers. Since neighbors are typically closer, network conditions are generally more favorable, offering higher bandwidth, lower latency, and reduced packet loss rates. Furthermore, exchanging parameters with neighbors does not require waiting for all participants to complete training, addressing the bottleneck issue observed in centralized FL systems.

We summarize the primary challenges of implementing decentralized federated learning in edge computing environments as follows:

- **Limited communication capabilities**: Edge networks often face constraints in bandwidth, latency, and reliability. The large model parameters typical of deep learning require substantial bandwidth for transmission, which can result in slowdowns and interruptions due to unstable network conditions and dynamic signal strength. In addition, the energy cost of prolonged communication can rapidly deplete the battery life of edge devices, damaging the user's quality of experience.
- **Data heterogeneity**: Edge devices collect data that are often non-IID (non independent and identically distributed). The local dataset varies significantly between devices due to unique user environments and local conditions. This heterogeneity can lead to biased local models that do not generalize well across other devices when exchanging models with others in federated learning, posing a challenge for achieving a consistent, high-performing global model. Effective decentralized federated learning must, therefore, be able to handle diverse data distributions while retaining high performance for local data.

To address these issues, we propose a solution with the following components: (1) To tackle data heterogeneity in distributed environments, we introduce a prototype propagation technique that aligns the representation of samples with the same label across heterogeneous environments, thereby enhancing model generalization. (2) For limited communication capacity, we propose a prototype exchange strategy that enables lightweight prototype transmission between multiple local training rounds instead of exchanging full model parameters, significantly reducing communication overhead. Additionally, we leverage the lightweight nature of prototype information to design a multi-hop propagation mechanism, facilitating more effective collaboration. The main contributions of our work are summarized as follows:

- **Distributed prototype learning for enhancing collaborative training**: We introduce a prototype learning mechanism to improve the generalization and effectiveness of collaborative training in decentralized federated learning (DFL) environments. By

aligning the representations of similarly labeled samples across neighboring devices, this method addresses data heterogeneity, reducing the negative impacts of non-IID data distributions and enhancing model adaptability to diverse local data characteristics.

- **Alternative distributed prototype exchange and parameter aggregation**: To address communication and energy limitations in edge computing, we propose a lightweight prototype exchange strategy, enabling edge devices to share minimal prototype information over multiple local training rounds instead of large, full model parameters. Additionally, our multi-hop propagation mechanism facilitates efficient, neighbor-based parameter sharing, mitigating the effects of network instability and enhancing scalability in edge networks.
- **Extensive experimental validation**: We perform comprehensive experiments to validate the effectiveness of our approach across various edge computing scenarios. Our experimental results demonstrate significant improvements in communication efficiency, model generalization, and system robustness, providing empirical evidence of the practical advantages and applicability of our proposed methods in real-world DFL settings.

The roadmap of this paper is organized as follows. Section 2 reviews the related literature, discusses the limitations of existing methods, and introduces a preliminary scenario. Section 4 presents the design details of our work, from its building components to the training algorithm. Section 5 analyzes the model performance via extensive experiments. Finally, concluding remarks and potential research directions are given in Section 6.

2. Related Work

Heterogeneity remains a significant challenge in federated learning (FL) [5–7], prompting extensive research into solutions tailored to various scenarios. The core goal of addressing heterogeneity is to achieve a global model with robust generalization across devices, exemplified by approaches like FedProx [8], SCAFFOLD [9], and FedNova [10]. To counteract model drift, local updates are adjusted to reduce discrepancies between local and global weights, thus optimizing model performance. While effective, this straightforward approach is limited, as a single global model may struggle to generalize effectively across devices with significant statistical heterogeneity. Recent advances in personalized federated learning (PFL) seek to enhance local model performance. Methods such as Per-FedAvg [11] and FedRep [12] focus on improving global model initialization, followed by fine-tuning for enhanced personalization on heterogeneous datasets. Meanwhile, pFedMe [13] and Ditto [14] frame global and local model optimization as two distinct tasks, effectively transforming local model optimization into a bilevel problem, albeit at the cost of increased complexity in FL optimization. Other methods, including FedAMP [15] and FedFomo [16], propose personalized aggregation schemes that preserve local device information during global aggregation, though they often overlook factors such as gradient misalignment, model heterogeneity, and network latency. Addressing the challenge of diverse data distributions and device models, FedProto [17] enhances generalization by sharing prototype representations across devices. Additionally, FedProto adds an L2 regularization term to the training objective to minimize the discrepancy between local and global representations, promoting personalization. Inspired by this approach, we integrate prototype representation learning into DFL to reduce communication overhead and balance personalization with generalization in heterogeneous environments.

To mitigate potential bottlenecks associated with centralized parameter servers and conserve communication resources for devices with limited capacity, decentralized distributed learning has been proposed [18,19]. In this approach, model parameters are

exchanged along network links based on the underlying network topology, eliminating the need for central aggregation and broadcasting by a parameter server. These consensus-based distributed optimization methods, developed from distributed averaging algorithms [20,21], offer rigorous convergence guarantees. The convergence rate of decentralized distributed learning depends in part on the communication topology [18,20]. Specifically, a larger spectral gap in the adjacency matrix associated with the communication topology correlates with a slower convergence rate. It follows that denser topologies, which exhibit smaller spectral gaps, yield faster convergence rates in terms of training epochs, though they may not provide advantages in wall-clock time.

Building on distributed learning frameworks, several studies have explored semi-decentralized learning methods [22,23]. These approaches apply centralized aggregation within local clusters, followed by peer-to-peer parameter exchange across clusters after in-cluster aggregation. To further reduce network traffic and accelerate training, [24] incorporates compression operators to improve communication efficiency in device-to-device interactions. For non-IID datasets, DFL-PENS [25] utilizes random gossip communication to identify neighboring devices with similar data distributions, facilitating collaborative learning among them. Similarly, DeceFL [26], a fully decentralized federated learning framework, enables local models to interact solely with their neighbors, making it effective in time-varying topology and non-IID data environments. However, the effectiveness of these DFL methods diminishes as data volume decreases or distributional differences increase. This decline primarily arises from transmitting gradients or model parameters, which intensifies the communication burden among devices. Additionally, simple aggregation struggles to achieve optimal results due to gradient misalignment, and these methods may not be well suited to highly heterogeneous environments.

3. System Model and Preliminary

3.1. From Centralized Machine Learning to Federated Learning

Machine learning models include a set of parameters learned based on training data. A training data sample indexed by j usually consists of input $\mathbf{x}_j$ and expected output y_j, which is also known as a label. To facilitate learning, each model has a loss function defined on its parameters $\mathbf{w}$ (we use a single $\mathbf{w}$ for simplicity and without loss of generality) for each data sample (x_j, y_j). The loss function calculates the error of the model on the training data, and the model learning process minimizes the loss function using an optimizer (e.g., mini-batch gradient descent). For each data sample j, we define the loss function as $\mathcal{L}(f(\mathbf{w}; \mathbf{x}_j), y_j)$, where $\mathbf{w}$ is the model's parameters.

Assume that we have N edge nodes with local datasets $\mathcal{D} \triangleq \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_i, \ldots, \mathcal{D}_N\}$. For each dataset $\mathcal{D}_i$ at node i, the loss function on the collection of data samples at this node is

$$F_i(\mathbf{w}) \triangleq \mathbb{E}_{(\mathbf{x}_j, y_j) \sim \mathcal{D}_i} \mathcal{L}(f(\mathbf{w}; \mathbf{x}_j), y_j) \tag{1}$$

Based on the local loss function, now, we can define the global loss function on all the distributed datasets as

$$F(\mathbf{w}) \triangleq \frac{\mathcal{D}_i}{|\mathcal{D}|} F_i(\mathbf{w}). \tag{2}$$

The objective of federated learning is to minimize the above loss function to obtain a unified model parameter:

$$\mathbf{w} = \arg\min_{\mathbf{w}} F(\mathbf{w}). \tag{3}$$

Note that $F(\mathbf{w})$ cannot be directly computed without sharing information among multiple nodes, thus protecting the participant's privacy.

In practice, a centralized parameter server exists for participants, nodes, or users to upload their own parameters and obtain the global parameters for the next round of training. This paradigm is called centralized federated learning. However, as we have discussed before, uploading their parameters to a cloud server is not preferable due to the unstable internet connection, high latency, and low bandwidth nature of edge nodes. Instead, if one could share its own parameters only with one's network neighbors, then we can bound the bandwidth requirement by restricting the number of its neighbor. Also, the peer-to-peer style makes it much easier for the system to expand its scale.

3.2. Decentralized Federated Learning

Next, we consider the parameter update process in decentralized federated learning. At the beginning of round k, we denote the parameter hold by node i as $\mathbf{w}_i^{(k)}$. By setting $\mathbf{w}_i^{(k,0)} = \mathbf{w}_i^{(k)}$, the node i updates it local parameters by gradient descent:

$$\mathbf{w}_i^{(k,\tau'+1)} = \mathbf{w}_i^{(k,\tau')} - \eta \nabla F_i(\mathbf{w}_i^{(k,\tau')}), \quad \forall \tau \quad 0 \leq \tau < \tau, \tag{4}$$

where η indicates the learning rate.

After finishing local updating, the node sends its model parameters to neighbors. Meanwhile, the node also receives parameters from its neighbor and updates its own parameters:

$$\mathbf{w}_i^{(k,t+\frac{1}{2})} = \sum_{j=1}^{n} W_{ij} \mathbf{w}_i^{(k,t)} \tag{5}$$

$$\mathbf{w}_i^{(k,t+1)} = \mathbf{w}_i^{(k,t+\frac{1}{2})} - \eta \nabla F_i(\mathbf{w}_i^{(k,t)}) \tag{6}$$

3.3. Theoretical Analysis

Assumption 1 (L-Smoothness). *Each local objective function $f_i : \mathbb{R}^d \to \mathbb{R}$ on workers is L-smooth:*

$$\|\nabla f_i(y) - \nabla f_i(x)\|^2 \leq L\|y - x\|^2, \quad \forall x, y \in \mathbb{R}^d, \ \forall i \in \mathcal{M}. \tag{7}$$

Assumption 2 (Bounded Gradient Variance). *The variance of stochastic gradients at each worker is bounded:*

$$\mathbb{E}\|\nabla F_i(x; i) - \nabla f_i(x)\|^2 \leq \sigma^2, \quad \forall x \in \mathbb{R}^d, \ \forall i \in \mathcal{M}, \tag{8}$$

$$\frac{1}{n}\sum_{i=1}^{n} \|\nabla f_i(x) - \nabla f(x)\|^2 \leq \zeta^2, \quad \forall x \in \mathbb{R}^d, \ \forall i \in \mathcal{M}. \tag{9}$$

Assumption 3 (Spectral Gap). *The graph weight matrix W is symmetric and doubly stochastic. We define $\rho = \max\{|\lambda_2(W)|, |\lambda_n(W)|\}$ and assume $\rho < 1$. $\lambda_i(W)$ means the i-th smallest eigenvalue of W.*

Remark 1. *In decentralized federated learning, we typically use the adjacency matrix A to represent the network topology. However, we cannot directly use A because the weight matrix W needs to be a doubly stochastic matrix (i.e., each row and column sums to 1). To address this, we apply a transformation to convert A into a doubly stochastic matrix.*

We use the Sinkhorn–Knopp algorithm (presented in Algorithm 1), an iterative normalization method that scales the rows and columns of a matrix to sum to 1. Given an adjacency matrix A, the algorithm iteratively normalizes its rows and columns to approximate a doubly stochastic matrix W.

Algorithm 1 Sinkhorn–Knopp algorithm for doubly stochastic matrix.

Require: Adjacency matrix $A \in \mathbb{R}^{n \times n}$, tolerance ϵ
Ensure: Doubly stochastic matrix W
Initialize $W \leftarrow A$
while not converged **do**
  for $i = 1$ to n **do** ▷ Row normalization
    $W_{i,:} \leftarrow W_{i,:} / \sum_j W_{i,j}$
  end for
  for $j = 1$ to n **do** ▷ Column normalization
    $W_{:,j} \leftarrow W_{:,j} / \sum_i W_{i,j}$
  end for
  if $\|W\mathbf{1} - \mathbf{1}\| < \epsilon$ and $\|W^T\mathbf{1} - \mathbf{1}\| < \epsilon$ **then**
    break
  end if
end while

As we can learn from the algorithm, we initialize W with the adjacency matrix A. The algorithm then alternates between normalizing the rows and columns until W is approximately doubly stochastic. Convergence is achieved when the row and column sums are sufficiently close to 1, within a specified tolerance ϵ.

Lemma 1. *Based on the above assumptions, with $\eta \leq \frac{1}{4L}$, we have the following [18,27]:*

$$\mathbb{E}\left[f(\bar{x}^{(k+1)})\right] \leq f(\bar{x}^{(k)}) \tag{10}$$

$$- \frac{\eta}{4}\left\|\nabla f(\bar{x}^{(k)})\right\|_2^2 \tag{11}$$

$$+ \frac{\eta L^2}{m}\sum_{i=1}^{m}\left\|\bar{x}^{(k)} - x_i^{(k)}\right\|_2^2 \tag{12}$$

$$+ \frac{\sigma^2 \eta^2 L}{m}. \tag{13}$$

We can observe that the gap between the expected model parameter $\mathbb{E}\left[f(\bar{x}^{(k+1)})\right]$ next round and the current model parameter can be bounded. Typically, when the model is about to converge, the gradient term will be small, and the average model parameter will be close to each client's parameter. As these two terms converge to zero, only a small perturbation in parameters will be observed between two consecutive rounds, depending on learning rate n, etc., indicating the model's convergence and performance saturation.

4. Solution

We compare our proposed decentralized federated learning algorithm with existing mainstream paradigms as in Figure 1. In our proposed paradigm, we alternately exchange model parameters and prototypes among clients in a fully decentralized manner. Tradi-

tional federated learning approaches typically focus on exchanging model parameters, which can lead to issues such as slow convergence and increased communication overhead, especially in non-IID scenarios. Some methods attempt to incorporate prototype exchanges, but they often rely on centralized coordination, limiting their scalability and effectiveness. In contrast, our method allows each client to autonomously share both parameters and prototypes with neighbors, fostering collaborative learning while preserving data privacy. This dual exchange mechanism not only enhances the model's performance by leveraging richer information but also improves convergence speed and robustness against data heterogeneity, positioning our approach as a more efficient solution in decentralized learning contexts.

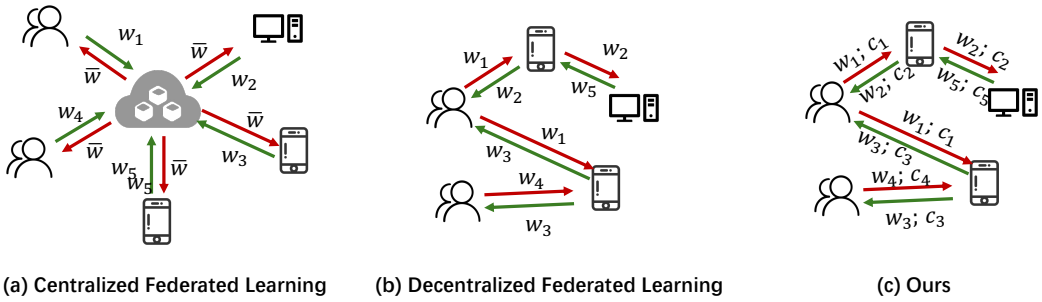


Figure 1. Comparison of federated learning paradigm.

4.1. Prototype Learning

In this section, we present our proposed approach, PearFL, in detail. Based on previous discussions, the main challenge we need to address is stabilizing model learning in a distributed setting where data are skewed and heterogeneous across clients.

Challenges and difficulties: Specifically, we focus on model M, where the inference process of M can be divided into multiple layers. Without loss of generality, we split the model inference into two stages: $f(g(\mathbf{x}))$. Here, for most models, $g(\mathbf{x})$ is referred to as the encoder, and the intermediate representation $z = g(\mathbf{x})$ is generated. The function $f(\cdot)$ represents the downstream task (e.g., in classification tasks, $f(\cdot)$ is the classification head). As observed in numerous studies, intermediate representations often exhibit structural properties: representations of the same class tend to be closer, while those of different classes are more distant. Even among similar classes, there is usually a larger cluster center distance than between dissimilar classes. This observation inspires us to refine the intermediate representations.

Observation: Due to the heterogeneity in training across different nodes, we observe that the intermediate representation distances for the same class often vary significantly across clients. There may even be conflicts: for instance, on client i, the representation of class A might be closer to the representation of class B on client j. When such models are aggregated, this can lead to blurred decision boundaries between classes A and B. This issue may propagate across nodes in a distributed federated learning setup, making the model convergence slow and may result in suboptimal performance.

Proposed solution: We begin by defining a prototype. A prototype represents the center of the image embeddings for a particular class, as computed by each participant's local model. We define a prototype $C(j)$ to represent the j-th class in the set C. For the i-th client, the prototype of class j is the mean of the embedding vectors of instances in class j:

$$C_i^{(j)} = \frac{1}{|\mathcal{D}_{i,j}|} \sum_{(\mathbf{x},y)\in\mathcal{D}_{i,j}} g_i(\phi_i;\mathbf{x}), \tag{14}$$

where $\mathcal{D}_{i,j}$ is a subset of the local dataset $\mathcal{D}_i$, consisting of training instances belonging to class j.

After obtaining these prototypes, we introduce a constraint during the local training of each participant, encouraging the model to generate similar representations for the same class, thereby reinforcing the cluster structure of intermediate representations and enhancing downstream task performance. Specifically, we modify the local loss function as follows:

$$\mathcal{L}' = \mathcal{L}(f(\mathbf{w}; \mathbf{x}_j), y_j) + \lambda \| g(\mathbf{x}_j) - C_i^{(j)} \|_2^2, \tag{15}$$

where $\mathcal{L}$ is the original task loss, and λ is a hyperparameter that controls the strength of the "clustering constraint" by minimizing the L_2 distance between the intermediate representation $g(\mathbf{x}_j)$ and the prototype $C_i^{(j)}$. If all the participants share the same $C^{(j)}$ for all classes j, there will be improved consistency in class representations across participants, facilitating better alignment in the collaborative learning process. This shared understanding of class prototypes enhances the global model's robustness, accelerates convergence by reducing discrepancies between local models, and, therefore, improves overall model performance. However, in decentralized federated learning, $C_i^{(j)}$ may not be the same; stabilizing the $C_i^{(j)}$ and making it consistent under distributed settings is critical, which is our focus in the next part.

4.2. Distributed Prototype Exchange and Propagation

In contrast to centralized federated learning, where the central server can aggregate all devices' parameters and their prototypes, in decentralized federated learning, only information exchange with neighbors is allowed. To address this, we propose a distributed prototype exchange and propagation algorithm for improved aggregation of knowledge across nodes.

In Algorithm 2, each client maintains a dictionary of prototypes. Each key in this dictionary is a label, and each value is an embedding anchor representing the prototype for that class. The algorithm aggregates prototypes from neighbors based on the topology matrix, computing a weighted average for each class. Prototype exchanges provide beneficial properties in decentralized learning. When a client has limited knowledge about a particular class, it relies more heavily on its neighbors' prototypes during aggregation. The existence of prototypes helps prevent clients with limited data for certain classes from deviating due to optimization on other classes. This enables clients to retain information for less-represented classes and implicitly preserves knowledge gains from other clients. As a result, the model's generalization ability and convergence speed are improved.

Inspired by node feature propagation in graph neural networks [28], we extend this approach by repeating the prototype exchange process multiple times. This iterative propagation allows prototypes to gradually align across a larger scope of nodes, facilitating a more comprehensive representation of class structures throughout the network. Since prototypes are lightweight compared to full model parameters, this transmission process is much faster. It enables us to add extra communication rounds between transmitting heavyweight model parameters. This alternating exchange effectively balances the communication load and accelerates convergence in decentralized learning.

Algorithm 2 Distributed prototype aggregation.

Require: Prototype dictionaries $C_i^{(j)}$, sample counts $|D_{i,j}|$, topology matrix W
Ensure: Aggregated prototypes $C_i^{(j)}$ for each client

Initialize $C_i^{(j)}$ as empty dictionaries for each client i
for each client i **do**
  Find neighbors of i using W
  Initialize `cache` to store prototypes from neighbors
  for each neighbor k of i **do**
    for each class j in $C_k^{(j)}$ **do**
      if $j \notin$ `cache` **then**
        Initialize `cache`$[j]$ with empty lists for prototypes and weights
      end if
      Append prototype $C_k^{(j)}$ and weight $|D_{k,j}|$ to `cache`$[j]$
    end for
  end for
  for each class j in `cache` **do**
    Normalize `cache`$[j]$ weights such that sum to 1
    Initialize $C_i^{(j)} \leftarrow \mathbf{0}$ for weighted aggregation
    for each prototype $C_k^{(j)}$ in `cache`$[j]$ **do**
      Update $C_i^{(j)} \leftarrow C_i^{(j)} + \left(\frac{|D_{k,j}|}{\sum_{m \in \texttt{cache}[j]} |D_{m,j}|} \right) C_k^{(j)}$
    end for
  end for
end for
return $C_i^{(j)}$

4.3. Overall Algorithm Description

We observe that prototype exchange itself carries rich information about class structure: it indicates the inherent relationships among classes, which gradually become clearer during the optimization process. Therefore, we propose that instead of transmitting model parameters between clients, we can transmit prototypes over several local epochs. This approach is efficient and lightweight, as the size of prototypes is significantly smaller than the entire model. In particular, model performance typically improves by a large margin after multiple local epochs, especially during the early stages of training. Meanwhile, the model's representations of the same class tend to change rapidly in response to local updates. Consequently, if the model continually aligns itself with outdated prototypes over multiple local epochs, this may hinder its performance by enforcing a suboptimal representation. Motivated by this, we integrate our prototype exchange and propagation algorithm into a decentralized federated learning framework, allowing clients to update each other with more relevant and recent prototypes. The algorithm is designed to capture evolving representation structures across clients without the overhead of full model transmission. This process can be regarded as speedy and can easily be made asynchronous, in which participants are not required to wait for the new prototypes to update. Instead, the participant can replace the old prototypes when it receives the new one. Based on the above description, we formalize the training process as in Algorithm 3.

Algorithm 3 PearFL: Decentralized Federated Learning with Inter-Epoch Prototype Exchange.

Require: Initial model parameters w_i and prototypes $C_i^{(j)}$ for each client i, number of local epochs E, topology matrix W

Ensure: Trained model parameters w_i and updated prototypes $C_i^{(j)}$ for each client i

for each communication round $t = 1, 2, \ldots, T$ **do**
  for each client i in parallel **do**
    for local epoch $e = 1, 2, \ldots, E$ **do**
      Local Training: Perform one epoch of training on local model w_i using local data.
      Prototype Exchange: Execute the prototype exchange and aggregation process as described in Algorithm 2 to update $C_i^{(j)}$ for each class j.
    end for
  end for
end for
return Trained model parameters w_i and updated prototypes $C_i^{(j)}$ for each client i

4.4. Discussion on Robustness

The above system design mainly focuses on stable edge devices and network scenarios. We now discuss how our system can remain robust under an unstable environment.

- **Node join and leave:** Due to mobility and unstable network conditions, devices may not remain continuously connected to the federated learning system. For PearFL, if a node leaves the system, it will not cause any interruption or significant degradation to the ongoing training process. The decentralized structure and distributed model representations ensure that the remaining nodes can continue to update and exchange their prototypes without depending on a single participant. Similarly, when a new node joins, it can swiftly initialize its parameters after establishing network connections with its neighbors. This can be achieved by averaging the local models of those neighbors. This seamless join-and-leave capability ensures robustness and flexibility, maintaining efficient collaboration even under dynamic network conditions. Moreover, device failure can also be regarded as a device leave case.
- **Different connection quality:** Our framework is designed to operate efficiently under varying network conditions. By exchanging compact prototypes rather than large model parameters, PearFL reduces the communication load, making it more resilient to inconsistent or low-bandwidth links. Moreover, the coordinator can monitor the network and suggest some possible high-quality links to deliver their model to the client. If a link is, indeed, low-quality, we can consider closing this link temporarily. In addition, lossless data compressions may also help maintain data transmission ability in a weak network environment.
- **Network split:** Network partitioning poses a significant challenge to distributed learning systems. In our case, if a partition occurs, we may leverage nodes with connection to public network infrastructure as a bridge to connect the isolated groups, allowing prototypes and model updates to flow between them once again. Even if a group becomes fully isolated for some time, the model training process remains meaningful within that partition. Once connectivity is restored, the accumulated knowledge can be merged back into each local model again, ensuring that temporary isolation does not result in a permanent loss of progress.

4.5. Complexity Analysis

In this subsection, we analyze our decentralized federated learning framework's computational, memory, and communication complexities, as below:

- **Computational complexity:** The key additional computation in our framework is the creation of prototypes for each client. Unlike the training time forward and backward, prototype computation relies solely on a forward pass without gradient tracking, substantially reducing its computational intensity. Empirically, the overhead introduced by prototype computation is modest relative to the entire training process. For example, generating prototypes accounts for about 18.7% of the total training time on MNIST and CIFAR10, and about 28.4% on CIFAR100. Thus, the computational cost ensures that prototype computation remains acceptable for edge devices.
- **Memory complexity.** Memory constraints are a critical consideration on edge devices. Our method introduces only limited memory overhead beyond that required for standard local training. Specifically, we store only a small set of prototypes. Depending on the dataset, we only maintain a number of vectors that equals the number of classes on the order of 10 to 100. Specifically, it only occupies 40 KB–400 KB of memory, which we believe is lightweight. This additional storage is negligible compared to the memory footprint of the model parameters and the local dataset. Consequently, the memory requirements remain aligned with conventional federated learning approaches, imposing no significant additional burden on resource-constrained devices.
- **Communication complexity.** Federated learning is often limited by communication bandwidth. Our approach mitigates this by leveraging prototypes, which serve as compact summaries of local data distributions. Rather than transmitting full model parameters or high-dimensional gradients, clients exchange fewer, lower-dimensional prototypes. Although our framework supports multi-hop propagation, the transmitted data volume decreases overall. The efficient peer-to-peer distribution of these compact prototypes helps prevent bottlenecks and reduces the frequency and size of payloads, thereby improving communication efficiency.
- **Energy consumption.** While our primary focus is on computational and communication complexity, it is worth noting the implications for energy consumption. Since the additional computation (prototype generation) is lightweight, and the communication volume is significantly reduced, the energy cost per training round is not expected to substantially increase. In many cases, it may even decrease due to less frequent transmission of large payloads. Thus, the energy implications are in line with, or potentially more favorable than, standard decentralized federated learning methods.

5. Experiments

5.1. Experimental Setup

5.1.1. Datasets

We evaluate the proposed Pear-DFL and baseline algorithms on three benchmark datasets: MNIST, CIFAR-10, and CIFAR-100. See Table 1.

Table 1. Profile of datasets used in our evaluations.

Dataset	Number of Instances	Number of Classes	Number of Channel	Image Size
MNIST	60,000	10	1	28×28
CIFAR-10	60,000	10	3	32×32
CIFAR-100	60,000	100	3	32×32

MNIST consists of 60,000 training images and 10,000 testing images of handwritten digits across 10 classes. CIFAR-10 contains 50,000 training images and 10,000 testing images from 10 classes, while CIFAR-100 consists of 100 classes with the same number of images

for training and testing as CIFAR-10. Each dataset is split into non-IID subsets across clients for federated training to simulate realistic data distribution scenarios. We use p to denote the level of non-IID distribution:

- **For CIFAR-10 and MNIST:** The non-IID level p% ($p = 30, 40, 50, 60, 70$) specifies the proportion of samples on each worker belonging to a single class. In contrast, the remaining samples belong to other classes. For instance, if $p = 30$, then 30% of the samples on a given worker belong to one class, with the other 70% belonging to other classes. Note that the data are closer to an IID distribution when p is low. We denote the non-IID levels of CIFAR-10 and MNIST as 30, 40, 50, 60, and 70.
- **For CIFAR-100:** Following the setting in the recent literature, the non-IID level p% ($p = 0, 15, 30, 45, 60$) specifies the number of classes each worker lacks in their dataset. For example, when $p = 30$, each worker lacks 30% of the total classes in CIFAR-100, meaning that 30 classes are missing from each worker's local dataset. The setting $p = 0$ represents nothing missing in the distribution of classes, which is IID. We denote the non-IID levels of CIFAR-100 as 0, 15, 30, 45, and 60.

5.1.2. Metrics

We validate our model performance based on two main metrics: classification accuracy and communication efficiency. Classification accuracy measures the model's performance on test images data, while communication efficiency is quantified by the total communication rounds required to reach a target accuracy.

To evaluate classification accuracy, we compute the following metrics:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \tag{16}$$

5.1.3. Configuration

The experiments are conducted on a deep learning server with Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40 GHz and 30 GB RAM, running Ubuntu 22.04. The federated learning models are implemented in PyTorch v2.3.0, Python 3.12, and CUDA 12.1. For each experiment, the SGD optimizer is used with a default learning rate of 0.05 with momentum set to 0.9, and the batch size is set to 32 for all models.

Two neural network architectures, ResNet9 and VGG9, are used as the federated model backbones. VGG9 has 3.49 million parameters, and ResNet9 has 6.61 million parameters. We do not use the full-size model of ResNet and VGG, which is usually ResNet50 and VGG19. We re-implement a smaller version but strictly follow the core idea of the original model to match the input complexity of the datasets we use for evaluation. The number of local epochs is set to 2 by default.

5.1.4. Baselines

We compare the proposed Pear-DFL approach against four commonly used federated learning algorithms:

- **FedAVG** [29]: A standard federated learning algorithm that performs model averaging after each communication round.
- **FedProx** [8]: An extension of FedAVG that includes a proximal term to improve stability on non-IID data distributions.
- **SCAFFOLD** [9]: An algorithm that mitigates client drift by introducing control variates, thereby improving convergence on heterogeneous data.
- **FedDyn** [30]: A federated method that dynamically adjusts loss functions to enhance convergence.

- **PerFedAVG** [11]: This is a personalized federated learning method trained by model-agnostic meta-learning.

5.2. Performance Comparison (RQ1)

5.2.1. Peak Performance

The experiment results in Table 2 and Figure 2 illustrate the strong performance of PearFL across multiple datasets, highlighting its accuracy and robustness, particularly in comparison to centralized federated learning methods such as FedAVG, FedProx, FedDyn, and SCAFFOLD.

Table 2. Accuracy comparison of different federated learning methods on various datasets.

Dataset	PearFL	FedAVG	FedProx	FedDyn	SCAFFOLD	PerFedAVG
MNIST	98.73	98.86	98.91	97.41	97.01	95.55
CIFAR-10	88.17	86.32	84.96	86.03	79.90	85.72
CIFAR-100	63.07	63.73	48.13	50.66	32.88	49.61

On the MNIST dataset, PearFL achieves an accuracy of 98.73%, nearly matching FedAVG's 98.86% and FedProx's 98.91%. This similarity in performance suggests that PearFL is highly effective even in a decentralized setting, closely aligning with traditional centralized approaches when handling simpler datasets. Moving to the CIFAR-10 dataset, PearFL achieves 88.17% accuracy, a result that is both competitive with FedAVG's 89.48% and notably higher than the performance of FedProx, FedDyn, and SCAFFOLD, which achieve 84.96%, 86.03%, and 79.90%, respectively. PerFedAVG performs slightly worse than FedAVG, which might be due to the challenging non-IID setting in our experiment. This demonstrates that PearFL maintains high accuracy even with more complex image data, indicating its robustness and adaptability. The decentralized structure of PearFL does not compromise its ability to capture important features in more challenging datasets, underscoring its viability as an alternative to centralized approaches.

In the CIFAR-100 dataset, which includes greater diversity and complexity, PearFL attains 63.07% accuracy. This performance is close to FedAVG's 65.12% and FedProx's 64.63%, while significantly outperforming FedDyn's 50.66% and SCAFFOLD's 32.88%. The substantial performance gap among PearFL, FedDyn, and SCAFFOLD may be caused by server state design. In particular, the update direction of the server state can be overwhelmed by high noise brought by averaging model parameters from data heterogeneous clients. For personalized meta-learning-enabled federated learning, we find it hard to deal with imbalanced dataset class distribution. This is because PerFedAVG is not directly optimized for heterogeneous data but for personalization. In addition, PerFedAVG cannot benefit explicitly from knowledge of few or even unseen samples. By comparing PearFL with these algorithms, we can learn that our decentralized approach has innate advantages for learning a well-generalized model across heterogeneous clients.

These results demonstrate the advantages of PearFL's decentralized approach. Despite the lack of a central server, PearFL achieves accuracy levels that rival those of centralized federated learning methods, suggesting that it can effectively coordinate and aggregate learning across distributed devices. This makes it a highly suitable solution for scenarios where limited central control or privacy requirements are crucial. Furthermore, PearFL's consistent performance across datasets with varying complexity, from MNIST to CIFAR-100, demonstrates its generalizability and adaptability. This resilience against increasing dataset complexity confirms PearFL's capability to handle a range of learning tasks effectively.

As we learn from the figures, we find that PearFL has a very competitive performance on all three datasets and only slightly lags behind three centralized federated learning

methods. It is because we select all clients to update in each round, while PearFL can only talk to its neighbors. According to [18], centralized federated learning is a special case of decentralized learning on the fully connected graph, and its spectral gap of communication topology is smaller than our sparse communication topology, which yields a faster convergence rate and better performance. However, we find that the performance gap is only marginal, showing that our scheme successfully achieves high accuracy. We will show that, in fact, under different network environments, our solution is much faster in convergence compared with the centralized counterparts.

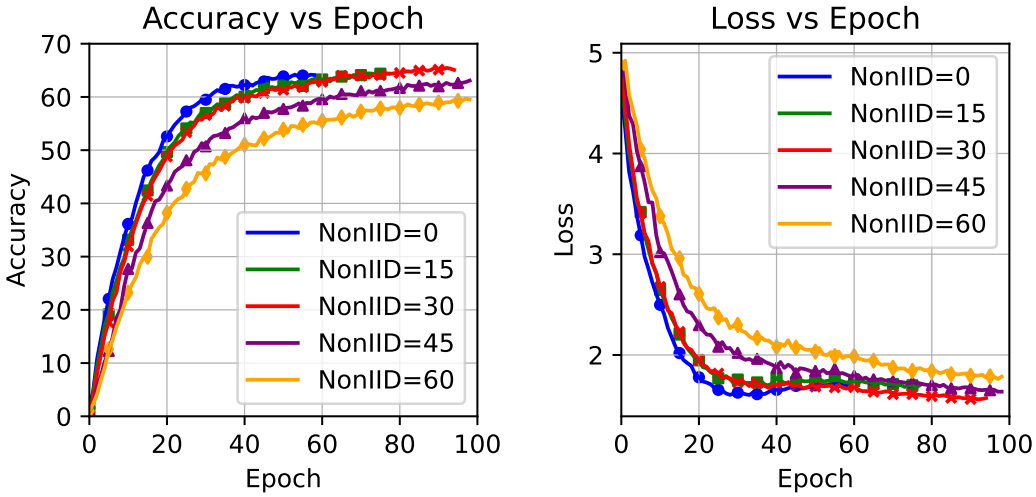


Figure 2. Performance under varied non-IID level on CIFAR-100 dataset.

5.2.2. Convergence Speed

In this experiment, we compare the convergence performance of three federated learning baselines: PearFL, FedAvg, and FedProx on two benchmark datasets, CIFAR-10 and CIFAR-100. Corresponding results are presented in Figure 3. We measure the accuracy and loss across epochs to assess their convergence behavior and generalization capabilities. For CIFAR-10, PearFL demonstrates faster convergence in accuracy than FedAvg and FedProx, achieving a higher accuracy plateau within 60 epochs. Similar trends are observed for CIFAR-100, where PearFL maintains superior accuracy growth, outperforming FedAvg and FedProx, particularly in the early epochs. On CIFAR-10, PearFL shows a rapid decrease in loss compared to the other algorithms, suggesting better convergence efficiency. FedAvg and FedProx exhibit slower decreases, indicating PearFL's improved robustness in optimizing model parameters. The loss patterns on CIFAR-100 confirm these observations, with PearFL achieving the lowest loss values more quickly than the alternatives. These results suggest that PearFL is more effective in both convergence rate and accuracy on the tested datasets. Future work could focus on exploring these algorithms' behavior in more complex scenarios and on additional datasets.

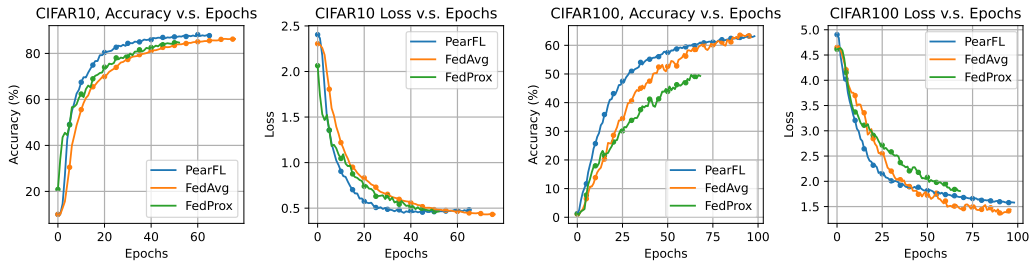
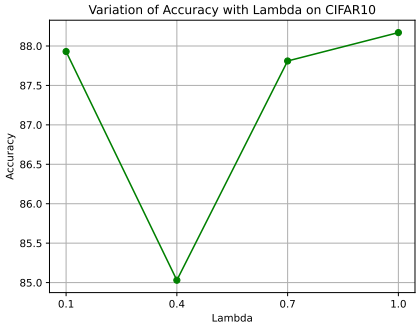


Figure 3. Convergence speed comparison.

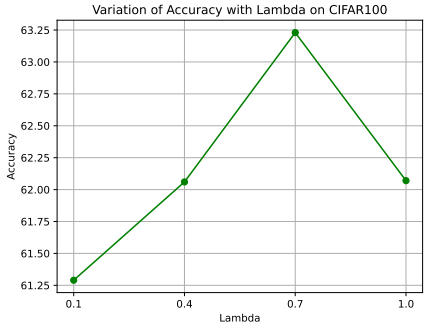
5.3. Hyperparameter Sensitivity (RQ2)

As we introduce an extra hyperparameter into the training loss function to calibrate the prototypes across distributed clients, finding the best hyperparameter value is critical for achieving the best learning performance. We conduct the experiments on two challenging datasets, CIFAR-10 and CIFAR-100, to better show the trends. In this experiment, we vary the hyperparameter λ to observe its impact on model accuracy for each dataset.

As shown in Figure 4a, for the CIFAR-10 dataset, accuracy initially starts at around 88% when $\lambda = 0.1$, drops significantly to 85% at $\lambda = 0.4$, and then recovers back to 88% as λ increases to 1. Similarly, in Figure 4b, for the CIFAR-100 dataset, we observe a different trend, where accuracy improves from 61.25% at $\lambda = 0.1$ to a peak of 63.5% at $\lambda = 0.7$ before declining again to around 61.5% at $\lambda = 1.0$. From these results, we conclude that a lower λ value in the loss function reduces the regularization penalty during training, which can lead to over-fitting. Conversely, a higher λ value may increase regularization strength, potentially resulting in under-fitting. It is essential for us to select a λ to trade-off the over-fitting and under-fitting performance. Consequently, we set λ to 1.0 for CIFAR-10 and λ to 0.7 for CIFAR-100.



(**a**) Impact of λ on CIFAR-10 dataset.



(**b**) Impact of λ on the CIFAR-100 dataset.

Figure 4. Impact of λ on CIFAR-10 and CIFAR-100 datasets.

5.4. Communication and Energy Analysis (RQ3)

We compare the communication efficiency of our PearFL algorithm against FedAvg and FedProx on CIFAR-10 and CIFAR-100 datasets. The corresponding tables are shown in Tables 3 and 4. For CIFAR-10, PearFL reaches 80% accuracy in just 20 rounds, totaling 208.0 s, outperforming FedAvg (38 rounds, 497.8 s) and FedProx (33 rounds, 336.6 s). On CIFAR-100, PearFL again demonstrates greater efficiency, completing 29 rounds and 275.5 s, whereas FedAvg requires 42 rounds (416.6 s). FedProx cannot reach 50% accuracy within 100 rounds, taking more than 1140 s. Our prototype propagation approach thus achieves faster convergence compared with centralized federated learning algorithms, making PearFL more communication-efficient than the alternatives, especially in complex datasets.

Table 3. Comparison of communication efficiency on CIFAR-10.

Algorithm	Comm. Round	Time Per Round	Total Time (Acc = 80%)
FedAvg	38	13.1 s	497.8 s
FedProx	33	10.2 s	336.6 s
PerFedAVG	90	15.2 s	1408.5 s
PearFL	20	10.4 s	208.0 s

Table 4. Comparison of communication efficiency on CIFAR-100.

Algorithm	Comm. Round	Time Per Round	Total
FedAvg	42	9.92 s	416.6 s
FedProx	100+	11.4 s	1140 s+
PerFedAVG	100+	14.8 s	1480 s+
PearFL	29	9.5 s	275.5 s

Regarding energy consumption, in our settings, all clients are running in best-effort mode to finish model training as soon as possible. Therefore, the overall consumption can be reduced if the communication time and communication rounds (equal to model training rounds) can be reduced. These can be inferred from the following two perspectives. For the computation, we can use $P = Wt$, with P, W, t representing total energy consumption, W representing device consumption, and t representing the training time. For communication, as we sometimes send prototypes instead of full model parameters in each round, the network traffic is reduced compared with other baselines. Moreover, for wireless communication, the energy consumption under the same bandwidth is mainly determined by communication time, which is proportional to the traffic size. In conclusion, our effort to reduce communication and training rounds can lead to less energy consumption for the training process across participants.

5.5. System Robustness

We investigate the system robustness from two different perspectives: network partitioning and node failure.

The first experiment investigates the impact of dynamic network topology changes and network partitioning on distributed training performance. Two experimental setups were designed to evaluate these effects. In the first setup, the network topology was completely switched every three epochs, as shown in the first column of Figure 5. In the second setup, the experiment alternated between five epochs of normal and five epochs of network partitioning. For the partitioning scenario, two configurations were considered: one with 10 nodes divided into two connected components (second column), and another with 20 nodes divided into three connected components (third column). Partitioning refers to splitting the network graph into isolated subgraphs that form independent connected components.

The results of the dynamic topology switching experiments (first column) show that the training process is robust to frequent changes in network topology. For CIFAR-10, the accuracy increases rapidly and converges to approximately 88% after 50 epochs. For CIFAR-100, while the convergence is slower due to the higher task complexity, the model still reaches an accuracy of around 60% after 40 epochs. These results demonstrate that dynamic topology switching has little impact on the model's ability to learn, highlighting the resilience of the distributed training process under such conditions.

In the network partitioning experiments (second and third columns), the results reveal a more noticeable effect on training performance. For CIFAR-10, when 10 nodes are partitioned into two components, the accuracy still improves steadily but exhibits periodic fluctuations corresponding to the partitioning cycles. These fluctuations indicate temporary disruptions caused by the partitions. When the number of nodes is increased to 20 and the graph is partitioned into three components, the accuracy curve becomes smoother, suggesting that the increased number of nodes helps mitigate the impact of partitioning. For CIFAR-100, the partitioning effects are more evident due to the higher task complexity. With 10 nodes and two partitions, the training process is slower, and periodic

fluctuations are more apparent. When 20 nodes are partitioned into three components, the fluctuations are reduced, and the training becomes more stable, ultimately reaching an accuracy of approximately 60%. Yet, the training process became slower with the increase in the number of nodes and the growth of partitioning size. Also, we learn that more nodes involved in the training may indicate a more stable training process.

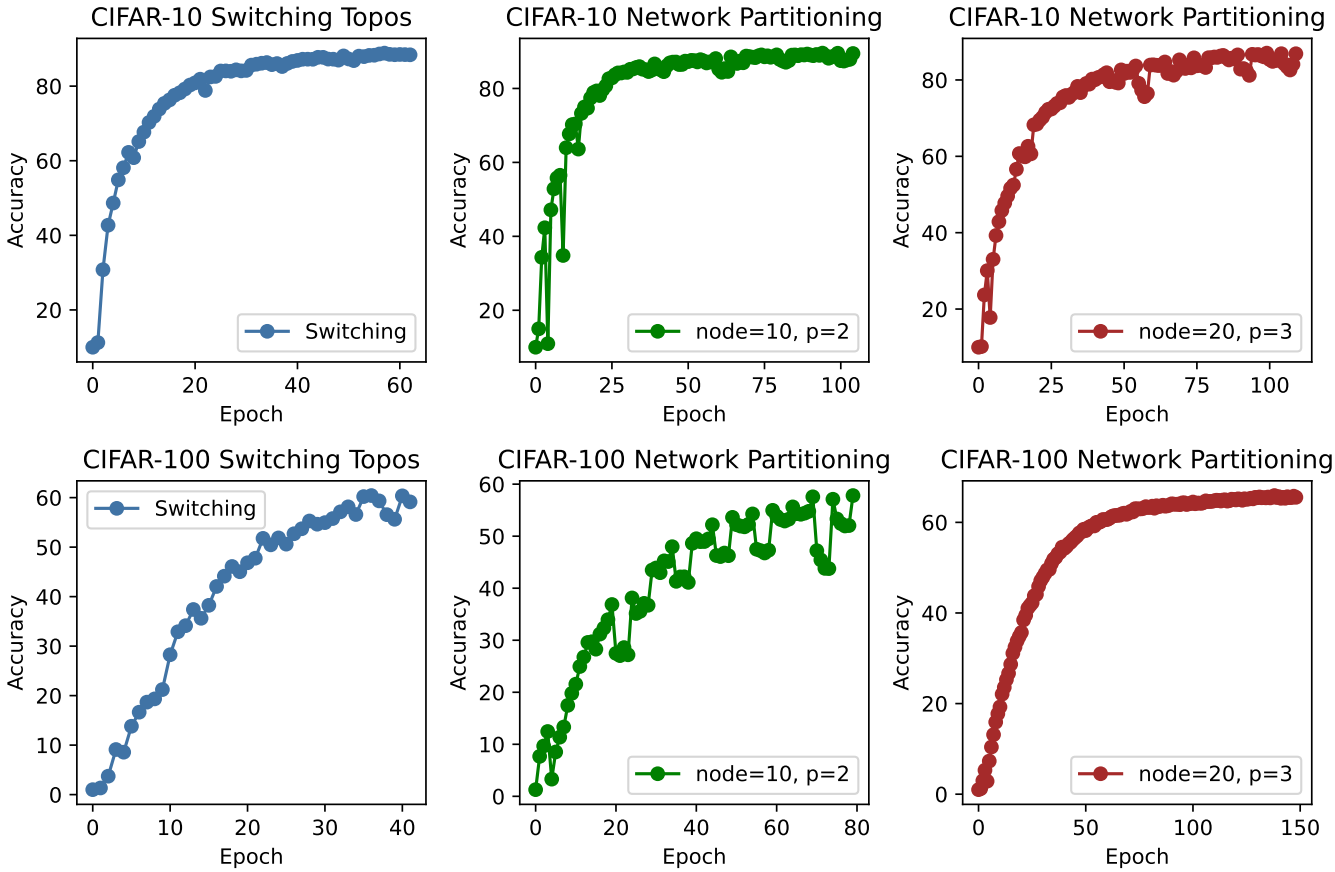


Figure 5. Training curves for dynamic network scenario.

The second experiment focused on node failure during the training process, and the training curves are presented in Figure 6. In our experiment, we have an environment with 20 nodes. Among them, 4 nodes (Node 4, Node 9, Node 13, and Node 17) were offline from epoch 5 to epoch 30, leading to huge performance gaps between them and the global model. However, once these nodes rejoined the system, their accuracies rapidly recovered and eventually caught up with the global model's accuracy, as we use the neighborhoods' model average to reinitialize the rejoin model. This indicates that our PearFL algorithm with proper join and leave design is highly robust against node failures, ensuring stable and efficient performance despite the temporary loss of several nodes.

In conclusion, we show that the proposed PearFL is robust to the dynamic network and device conditions—it will still converge, and the performance variation is not obvious.

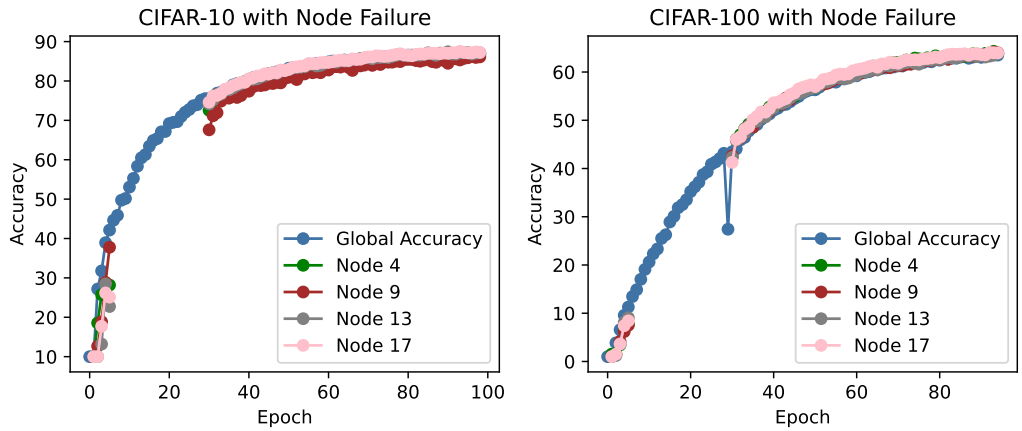


Figure 6. Training curves when node failure occurs.

5.6. Memory Footprint

The memory footprint comparison in Table 5 highlights the resource usage of various federated learning algorithms on both the server and client sides. On the server side, PearFL and FedAVG exhibit the lowest memory consumption at 240 MB and 251 MB, respectively, making them suitable for environments with limited server resources. In contrast, FedProx and FedDyn require significantly more memory, with 504 MB and 507 MB, respectively, which may pose challenges for deployment in resource-constrained settings. The reason why they take up more memory is that they maintain an extra parameter copy on both the server and client side as regularization and reference. PearFL, in contrast, does not need to store an extra parameter copy, and our local training process remains lightweight, similar to FedAVG. The only extra cost that arises is computing the prototypes.

Table 5. Average memory footprint comparison (in MB).

	FedAVG	FedProx	FedDyn	SCAFFOLD	PerFedAvg	PearFL
Server Memory	251	504	507	496	276	240
Client Memory	123	231	229	213	148	149

5.7. Transmission Latency

We evaluated the model size of two models. The ResNet9 model is approximately 42 MB, while the VGG9 model is about 13.5 MB. Our findings indicate that the model transmission time is highly sensitive to throughput. This conclusion is supported by the observation that increasing the bandwidth by 50 times leads to a nearly 40 to 50-fold reduction in transmission latencies, as shown in Table 6. We show that the communication latencies cannot be ignored and in some cases are even longer than the per-epoch update time; therefore, we again show the importance of reducing communication epochs by introducing prototype exchange.

Table 6. Model transmission latency under different settings (in seconds).

Network Environment	ResNet9	VGG9
Latency = 50 ms; throughput = 1 Mbps	353.08 s	112.45 s
Latency = 50 ms; throughput = 50 Mbps	7.16 s	2.35 s
Latency = 500 ms; throughput = 1 Mbps	353.98 s	113.35 s
Latency = 500 ms; throughput = 50 Mbps	8.06 s	3.25 s

6. Conclusions

This study presents PearFL, a decentralized federated learning framework that incorporates prototype exchange to address the unique challenges of edge computing environments. PearFL's lightweight prototype transmission and multi-hop propagation mechanisms enable efficient parameter sharing, reducing communication costs and enhancing model adaptability to heterogeneous data distributions. The experimental results validate PearFL's effectiveness in improving convergence speed, classification accuracy, and communication efficiency across multiple datasets. These findings highlight PearFL's potential to facilitate scalable and robust federated learning in decentralized settings. Future research directions include exploring PearFL's adaptability to more complex data distributions and investigating additional strategies to enhance model robustness in highly dynamic network conditions.

Author Contributions: Conceptualization, L.Q. and H.C. (Haoze Chen); methodology, L.Q.; software, L.Q., H.Z. and X.Z.; validation, S.C., X.Z. and H.C. (Hongyan Chen); formal analysis, S.C.; investigation, H.C. (Hongyan Chen); writing—original draft preparation, L.Q.; writing—review and editing, H.C. (Haoze Chen); visualization, H.C. (Hongyan Chen). All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Simonyan, K.; Zisserman, A. Very deep convolutional networks for large-scale image recognition. *arXiv* **2014**, arXiv:1409.1556.
2. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778.
3. Narayanan, A.; Ramadan, E.; Carpenter, J.; Liu, Q.; Liu, Y.; Qian, F.; Zhang, Z.L. A first look at commercial 5G performance on smartphones. In Proceedings of the Web Conference 2020, Taipei, Taiwan, 20–24 April 2020; pp. 894–905.
4. Yuan, X.; Wu, M.; Wang, Z.; Zhu, Y.; Ma, M.; Guo, J.; Zhang, Z.L.; Zhu, W. Understanding 5g performance for real-world services: A content provider's perspective. In Proceedings of the ACM SIGCOMM 2022 Conference, Amsterdam, The Netherlands, 22–26 August 2022; pp. 101–113.
5. Zhang, C.; Xie, Y.; Bai, H.; Yu, B.; Li, W.; Gao, Y. A survey on federated learning. *Knowl. -Based Syst.* **2021**, *216*, 106775.
6. Mammen, P.M. Federated learning: Opportunities and challenges. *arXiv* **2021**, arXiv:2101.05428.
7. Wen, J.; Zhang, Z.; Lan, Y.; Cui, Z.; Cai, J.; Zhang, W. A survey on federated learning: Challenges and applications. *Int. J. Mach. Learn. Cybern.* **2023**, *14*, 513–535.
8. Li, T.; Sahu, A.K.; Zaheer, M.; Sanjabi, M.; Talwalkar, A.; Smith, V. Federated optimization in heterogeneous networks. *Proc. Mach. Learn. Syst.* **2020**, *2*, 429–450.
9. Karimireddy, S.P.; Kale, S.; Mohri, M.; Reddi, S.; Stich, S.; Suresh, A.T. Scaffold: Stochastic controlled averaging for federated learning. In Proceedings of the International Conference on Machine Learning, PMLR, Virtual, 13–18 July 2020; pp. 5132–5143.
10. Wang, J.; Liu, Q.; Liang, H.; Joshi, G.; Poor, H.V. Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization. *arXiv* **2020**, arXiv.2007.07481. https://doi.org/10.48550/arXiv.2007.07481.
11. Fallah, A.; Mokhtari, A.; Ozdaglar, A. Personalized Federated Learning: A Meta-Learning Approach. *arXiv* **2020**, arXiv.2002.07948. https://doi.org/10.48550/arXiv.2002.07948.
12. Collins, L.; Hassani, H.; Mokhtari, A.; Shakkottai, S. Exploiting Shared Representations for Personalized Federated Learning. *arXiv* **2023**, arXiv.2102.07078. https://doi.org/10.48550/arXiv.2102.07078.
13. Dinh, C.T.; Tran, N.H.; Nguyen, T.D. Personalized Federated Learning with Moreau Envelopes. *arXiv* **2022**, arXiv.2006.08848. https://doi.org/10.48550/arXiv.2006.08848.
14. Li, T.; Hu, S.; Beirami, A.; Smith, V. Ditto: Fair and robust federated learning through personalization. In Proceedings of the International Conference on Machine Learning, PMLR, Virtual, 18–24 July 2021; pp. 6357–6368.
15. Huang, Y.; Chu, L.; Zhou, Z.; Wang, L.; Liu, J.; Pei, J.; Zhang, Y. Personalized Cross-Silo Federated Learning on Non-IID Data. *arXiv* **2021**, arXiv.2007.03797. https://doi.org/10.48550/arXiv.2007.03797.

16. Zhang, M.; Sapra, K.; Fidler, S.; Yeung, S.; Alvarez, J.M. Personalized Federated Learning with First Order Model Optimization. *arXiv* **2021**, arXiv.2012.08565. https://doi.org/10.48550/arXiv.2012.08565.
17. Tan, Y.; Long, G.; Liu, L.; Zhou, T.; Lu, Q.; Jiang, J.; Zhang, C. Fedproto: Federated prototype learning across heterogeneous clients. In Proceedings of the AAAI Conference on Artificial Intelligence, Online, 22 February–1 March 2022; Volume 36, pp. 8432–8440.
18. Lian, X.; Zhang, C.; Zhang, H.; Hsieh, C.J.; Zhang, W.; Liu, J. Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 5336–5346.
19. Sirb, B.; Ye, X. Decentralized consensus algorithm with delayed and stochastic gradients. *SIAM J. Optim.* **2018**, *28*, 1232–1254.
20. Xiao, L.; Boyd, S. Fast linear iterations for distributed averaging. *Syst. Control. Lett.* **2004**, *53*, 65–78.
21. Boyd, S.; Ghosh, A.; Prabhakar, B.; Shah, D. Randomized gossip algorithms. *IEEE Trans. Inf. Theory* **2006**, *52*, 2508–2530.
22. Lin, F.P.C.; Hosseinalipour, S.; Azam, S.S.; Brinton, C.G.; Michelusi, N. Semi-decentralized federated learning with cooperative D2D local model aggregations. *IEEE J. Sel. Areas Commun.* **2021**, *39*, 3851–3869.
23. Sun, Y.; Shao, J.; Mao, Y.; Wang, J.H.; Zhang, J. Semi-decentralized federated edge learning with data and device heterogeneity. *IEEE Trans. Netw. Serv. Manag.* **2023**, *20*, 1487–1501.
24. Liu, W.; Chen, L.; Zhang, W. Decentralized federated learning: Balancing communication and computing costs. *IEEE Trans. Signal Inf. Process. Over Netw.* **2022**, *8*, 131–143.
25. Onoszko, N.; Karlsson, G.; Mogren, O.; Zec, E.L. Decentralized federated learning of deep neural networks on non-iid data. *arXiv* **2021**, arXiv.2107.08517. https://doi.org/10.48550/arXiv.2107.08517.
26. Yuan, Y.; Liu, J.; Jin, D.; Yue, Z.; Chen, R.; Wang, M.; Sun, C.; Xu, L.; Hua, F.; He, X.; et al. DeceFL: A Principled Decentralized Federated Learning Framework. *arXiv* **2021**, arXiv.2107.07171. https://doi.org/10.48550/arXiv.2107.07171.
27. Wang, L.; Xu, Y.; Xu, H.; Chen, M.; Huang, L. Accelerating decentralized federated learning in heterogeneous edge computing. *IEEE Trans. Mob. Comput.* **2022**, *22*, 5001–5016.
28. Wu, Z.; Pan, S.; Chen, F.; Long, G.; Zhang, C.; Philip, S.Y. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* **2020**, *32*, 4–24.
29. McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; y Arcas, B.A. Communication-efficient learning of deep networks from decentralized data. In Proceedings of the Artificial Intelligence and Statistics, PMLR, Fort Lauderdale, FL, USA, 20–22 April 2017; pp. 1273–1282.
30. Acar, D.A.E.; Zhao, Y.; Navarro, R.M.; Mattina, M.; Whatmough, P.N.; Saligrama, V. Federated learning based on dynamic regularization. *arXiv* **2021**, arXiv:2111.04263.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.